

De Jackson 2 a Jackson 3 (sin errores de compilación)

1) Imports: `com.fasterxml.jackson` → `tools.jackson`

El core y el databind cambiaron de paquete raíz. Es el cambio que más líneas rompe.

Antes (Jackson 2)

```
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.SerializationFeature;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.json.JsonMapper;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.core.type.TypeReference;
```

Después (Jackson 3)

```
import tools.jackson.databind.ObjectMapper;
import tools.jackson.databind.JsonNode;
import tools.jackson.databind.SerializationFeature;
import tools.jackson.databind.DeserializationFeature;
import tools.jackson.databind.json.JsonMapper;
import tools.jackson.core.JsonException; // ya no JsonProcessingException
import tools.jackson.core.type.TypeReference;
```

Regla práctica: reemplaza `com.fasterxml.jackson.core / .databind` por `tools.jackson.core / .databind`. Pero no toques los imports de anotaciones (siguiente punto).

2) Anotaciones: `@JsonProperty` NO se mueve

El error más típico: mover también los imports de anotaciones. No lo hagas. Siguen en `jackson-annotations`, cuyo groupId sigue siendo `com.fasterxml.jackson.core` y cuyo paquete sigue siendo `com.fasterxml.jackson.annotation`.

// Se queda IGUAL en Jackson 2 y 3

```
import com.fasterxml.jackson.annotation.JsonProperty;
import com.fasterxml.jackson.annotation.JsonIgnore;
import com.fasterxml.jackson.annotation.JsonCreator;
import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.annotation.JsonFormat;
import com.fasterxml.jackson.annotation.JsonValue;
// El uso de @JsonProperty NO cambia nada
public class Cliente {
    @JsonProperty("nombre_completo")
    private String nombreCompleto;

    @JsonProperty("id_cliente")
```

```
private Long idCliente;
}
```

Lo único que sí se movió son las anotaciones de databind (menos usadas): `@JsonSerialize` y `@JsonDeserialize` pasan a `tools.jackson.databind.annotation`.

```
// antes
import com.fasterxml.jackson.databind.annotation.JsonSerialize;
// después
import tools.jackson.databind.annotation.JsonSerialize;
```

3) ObjectMapper → JsonMapper (ahora es immutable)

En Jackson 3 el mapper es immutable: se construye con un builder, ya no se configura con setters ni existe `.copy()`.

Antes (Jackson 2) — setters

```
ObjectMapper mapper = new ObjectMapper();
mapper.configure(SerializationFeature.INDENT_OUTPUT, true);
mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
mapper.registerModule(new JavaTimeModule());

ObjectMapper otro = mapper.copy(); // clonar y ajustar
```

Después (Jackson 3) — builder

```
JsonMapper mapper = JsonMapper.builder()
    .configure(SerializationFeature.INDENT_OUTPUT, true)
    .configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false)
    // JavaTime ya viene incluido: normalmente no hace falta addModule
    .build();

// .copy() ya no existe: se usa .rebuild()
JsonMapper otro = mapper.rebuild()
    .configure(SerializationFeature.INDENT_OUTPUT, false)
    .build();
```

Si inyectas el mapper de Spring, no lo construyas a mano: Spring Boot 4 ya te da uno de Jackson 3 configurado.

```
@Service
public class MiServicio {
    private final ObjectMapper mapper;
    public MiServicio(ObjectMapper mapper) { this.mapper = mapper; }
}
```

4) Excepciones y leer / escribir JSON

En Jackson 2, `JsonProcessingException` era checked (te obligaba a try/catch). En Jackson 3 todo cuelga de `JsonException`, que es unchecked. Ya no te obliga el compilador.

Antes (Jackson 2)

```
public String aJson(Cliente c) {
    try {
        return mapper.writeValueAsString(c);
    } catch (JsonProcessingException e) {
        throw new RuntimeException(e);
    }
}

public Cliente deJson(String json) throws JsonProcessingException {
    return mapper.readValue(json, Cliente.class);
}
```

Después (Jackson 3)

```
public String aJson(Cliente c) {
    return mapper.writeValueAsString(c); // ya no hace falta try/catch
}

public Cliente deJson(String json) {
    return mapper.readValue(json, Cliente.class); // ni throws
}
```

Los nombres de los métodos no cambian: `readValue`, `writeValueAsString`, `readTree`, `convertValue` siguen igual. Solo quitas los `throws` / `catch` (o los cambias por `catch (JsonException e)`).

```
JsonNode nodo = mapper.readTree(json);
List<Cliente> lista = mapper.readValue(json, new TypeReference<List<Cliente>>() {});
```

5) Serializador y deserializador personalizado

Cambian paquete y nombres: `JsonSerializer` → `ValueSerializer`, `SerializerProvider` → `SerializationContext`, y se quita el `throws IOException`.

Serializador — antes / después

Jackson 2

```
import com.fasterxml.jackson.core.JsonGenerator;
import com.fasterxml.jackson.databind.JsonSerializer;
import com.fasterxml.jackson.databind.SerializerProvider;
import java.io.IOException;
```

```

public class DineroSerializer extends JsonSerializer<BigDecimal> {
    @Override
    public void serialize(BigDecimal value, JsonGenerator gen, SerializerProvider provider)
        throws IOException {
        gen.writeString(value.setScale(2) + " MXN");
    }
}

```

Jackson 3

```

import tools.jackson.core.JsonGenerator;
import tools.jackson.databind.ValueSerializer;    // antes JsonSerializer
import tools.jackson.databind.SerializationContext; // antes SerializerProvider

public class DineroSerializer extends ValueSerializer<BigDecimal> {
    @Override
    public void serialize(BigDecimal value, JsonGenerator gen, SerializationContext ctxt) {
        gen.writeString(value.setScale(2) + " MXN"); // sin throws IOException
    }
}

```

Deserializador — antes / después

Jackson 2

```

import com.fasterxml.jackson.databind.JsonDeserializer;
import com.fasterxml.jackson.databind.DeserializationContext;

public class DineroDeserializer extends JsonDeserializer<BigDecimal> {
    @Override
    public BigDecimal deserialize(JsonParser p, DeserializationContext ctxt) throws IOException {
        return new BigDecimal(p.getValueAsString().replace(" MXN", ""));
    }
}

```

Jackson 3

```

import tools.jackson.databind.ValueDeserializer;    // antes JsonDeserializer
import tools.jackson.databind.DeserializationContext;

public class DineroDeserializer extends ValueDeserializer<BigDecimal> {
    @Override
    public BigDecimal deserialize(JsonParser p, DeserializationContext ctxt) {
        return new BigDecimal(p.getValueAsString().replace(" MXN", ""));
    }
}

```

6) Módulos (SimpleModule y JavaTime)

El soporte de java.time ya viene incluido: no necesitas registrar JavaTimeModule ni la dependencia jackson-datatype-jsr310. Y el módulo se agrega en el builder, no con registerModule sobre un mapper ya creado.

Jackson 2

```
SimpleModule m = new SimpleModule();
m.addSerializer(BigDecimal.class, new DineroSerializer());
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(m);
mapper.registerModule(new JavaTimeModule());
```

Jackson 3

```
SimpleModule m = new SimpleModule();
m.addSerializer(BigDecimal.class, new DineroSerializer());
JsonMapper mapper = JsonMapper.builder()
    .addModule(m)          // JavaTime ya está incluido
    .build();
```

7) API de streaming (JsonFactory / JsonParser / JsonGenerator)

Aquí hay varios renombres de métodos y tokens.

Jackson 2

```
import com.fasterxml.jackson.core.JsonFactory;
import com.fasterxml.jackson.core.JsonToken;

JsonFactory factory = new JsonFactory();
JsonParser p = factory.createParser(json);
while (p.nextToken() != null) {
    if (p.currentToken() == JsonToken.FIELD_NAME) {
        String campo = p.currentName();
    }
}
Object v = p.getCurrentValue();
gen.writeObject(obj);
```

Jackson 3

```
import tools.jackson.core.JsonFactory;
import tools.jackson.core.JsonToken;

JsonFactory factory = JsonFactory.builder().build(); // antes new JsonFactory()
JsonParser p = factory.createParser(json);
while (p.nextToken() != null) {
```

```

    if (p.currentToken() == JsonToken.PROPERTY_NAME) { // antes FIELD_NAME
        String campo = p.currentName();
    }
}
Object v = p.currentValue(); // antes getCurrentValue()
gen.writePOJO(obj);          // antes writeObject(obj)

```

8) Tabla resumen de renombres

Jackson 2	Jackson 3
JsonSerializer	ValueSerializer
JsonDeserializer	ValueDeserializer
SerializerProvider	SerializationContext
JsonProcessingException	JacksonException (unchecked)
JsonGenerationException	StreamWriteException
JsonGenerator.writeObject()	JsonGenerator.writePOJO()
JsonParser.getCurrentValue()	JsonParser.currentValue()
JsonToken.FIELD_NAME	JsonToken.PROPERTY_NAME
TypeFactory.defaultInstance()	TypeFactory.createDefaultInstance()
new JsonFactory()	JsonFactory.builder().build()
mapper.copy()	mapper.rebuild().build()
com.fasterxml.jackson.core/.databind	tools.jackson.core/.databind
com.fasterxml.jackson.annotation	(igual, no cambia)

La mayoría de estos renombres ya tienen ejemplo en las secciones anteriores. Los dos que solo aparecen aquí van con su ejemplo a continuación.

JsonGenerationException → StreamWriteException

Además de cambiar de nombre, ahora es unchecked (cuelga de JacksonException).

Jackson 2 — checked

```

try {
    gen.writeString(valor);
} catch (JsonGenerationException e) {
    throw new RuntimeException(e);
}

```

Jackson 3 — StreamWriteException, ya no obliga try/catch

```

gen.writeString(valor);

```

TypeFactory.defaultInstance() → createDefaultInstance()

Jackson 2

```
JavaType tipo = TypeFactory.defaultInstance()  
    .constructCollectionType(List.class, Cliente.class);
```

Jackson 3

```
JavaType tipo = TypeFactory.createDefaultInstance()  
    .constructCollectionType(List.class, Cliente.class);
```